Awaaz Shield

Final report: what was built, what was tested, and what is left

Prepared for Cheraag · 16 October 2026

1. Summary

Awaaz Shield is a working scam-call detection dashboard. It turns what is said on a call into text, scores the call from 0 to 100 line by line, and shows an operator the risk with a suggested action. The operator then flags, escalates or dismisses the call. The app never acts on its own.

In your own live test the call worked and the risk level was judged carefully, but the transcript text was not very accurate. This round therefore ended with settings aimed at transcript accuracy (section 4). Real-world accuracy of those settings is the main thing still to be tested.

At a glance	Status
Dashboard, scoring and operator actions (flag, escalate, dismiss, restore)	Works
Live audio from a microphone, a virtual cable or a browser tab	Works (your live test)
Speakerphone call with two voices on one microphone	Built and tested with a stand-in; needs a real-audio test
Two-person in-app call (two phones, no carrier)	Built; needs a test on two real phones
Ordinary mobile-network call (phone 1 to phone 2)	Not possible for free: needs a paid telephony provider
TinyFish web-agent integration	Removed completely

2. What the app does

Area	What it does
Risk score	Every caller line is scored 0 to 100 by a rules engine that reads English and Hinglish. It looks at the shape of a scam tactic (OTP request, KYC pretext, urgency, fake authority, safe-account transfer), ignores warnings such as "never share your OTP", and ignores the agent's own lines. Claude scoring can be switched on with an API key.
Risk levels	Low, medium, high and critical, with a suggested action for each level (see the bands below).
Operator actions	Flag puts a call in the Flagged tab. Escalate moves it to the top. Dismiss files it in Operator history, where it can be read and restored.
Views	Live monitor (call list, summary gauge, risk over time, transcript), Command center (every call at a glance) and Insights (risk mix, 3D skyline, trajectories).
Samples	Seven sample calls load with one click, covering critical, high, medium and low.
Sign-in and storage	Firebase sign-in and Firestore storage when configured; a development login otherwise. Numbers such as OTPs and card numbers are hidden in what is saved, and audio is never stored.
Languages	Interface in English and seven Indian languages (Hindi, Bengali, Gujarati, Kannada, Marathi, Tamil, Telugu).

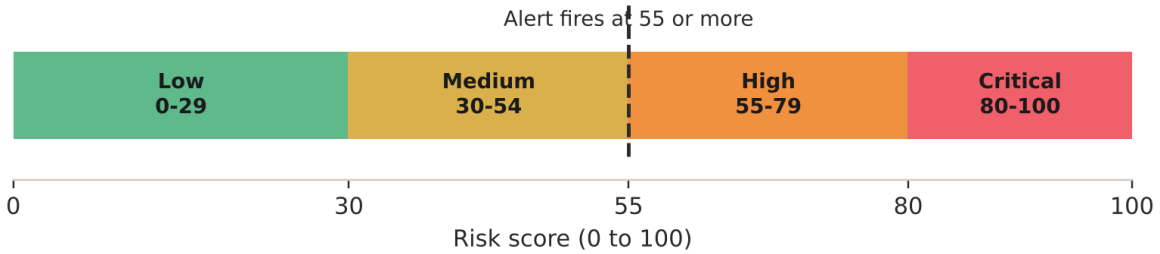


Figure 1. Risk levels. An alert fires at a score of 55 or more (the high and critical bands).

3. How a call is evaluated

Whatever the source, the audio takes the same path: the backend relays it to Deepgram (the key never reaches the browser), each finished phrase becomes a transcript line, the scorer rates it, and the dashboard updates.

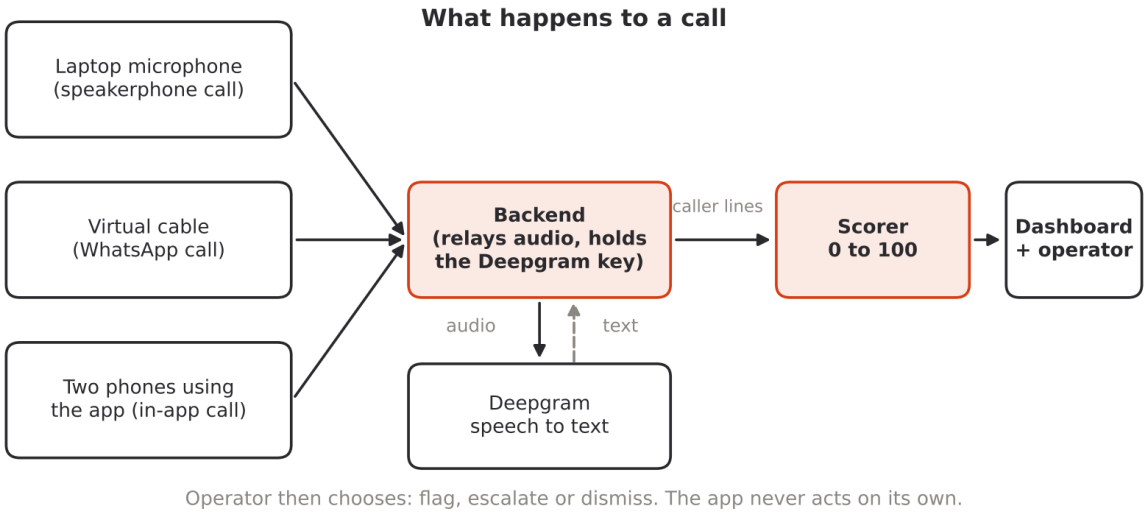


Figure 2. The path of a call from audio source to operator.

Ways to bring a call in

Route	How it works	Cost	Status
A. Laptop microphone	Put the call on speaker near the laptop, or speak into it. The browser streams the microphone to the backend.	Free	Works (your live test)
B. Speakerphone, two voices	Same as A, with "Two voices on this audio" ticked. Deepgram separates the voices; the first voice heard is the agent, the other is the caller. A Swap button fixes reversed roles.	Free	Built; tested with a stand-in server only
C. In-app call, two phones	One person starts a call and gets a 6-character code; the other joins. Each phone sends only its own microphone, so caller and agent are separate. Both must accept a consent notice first.	Free	Built; needs a test on two phones
D. WhatsApp or other app call	Send the call's sound to a virtual cable (for example BlackHole) and pick it as the audio input, or share a browser tab's audio.	Free	Works in principle; not tried with a real call
E. Ordinary mobile call, phone 1 to phone 2	The audio must pass through a number run by a telephony provider (Twilio, Exotel and similar). Apps cannot record or tap such calls on iOS or Android.	Paid	Not possible for free

4. What changed in this round

Change	Details
TinyFish removed	The web-agent "Check the caller's claim" feature is gone: backend module and test, the verify route, the side panel, all settings and keys, the status row, the consent-screen item, the strings in all eight languages, and the terms text. A final search finds no remaining reference in the source.
New design	Charcoal background, orange-red accent, gold and soft red for risk, following your screenshots (Figure 3). Flat cards and one clean font.
3D shield	Pulsing inner core, glowing halo, two particle rings turning in opposite directions, orbiting sparks, a scan line, a faster beat as risk rises, and a shockwave ring whenever the risk level changes. Larger on the call summary.
Particle wave	Reacts to the pointer, sends a swell across when the risk level changes, and its crests drift between two colours. The login shield is recoloured to ember tones.
Hover glow and background	A soft light follows the pointer over cards and lights their edge in the current risk colour. Soft colour blobs drift behind the workspace and shift to green, gold, orange or red with the call's risk.
Speaker separation	New option for speakerphone calls: Deepgram labels each voice, the first voice heard is the agent, and a Swap button reverses the roles.
Transcript accuracy	Scam-call vocabulary boost (OTP, KYC, UPI PIN, CVV, Aadhaar, RBI, SBI, HDFC, AnyDesk and about 20 more), a language choice (auto English plus Hindi, English India, Hindi), noise suppression and automatic volume for real microphones (kept off for virtual cables), and a phrase now closes after 0.5 s of silence instead of 0.3 s.

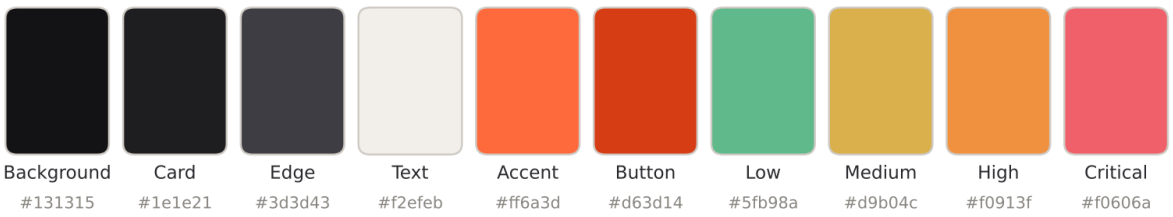


Figure 3. Colour palette used across the app.

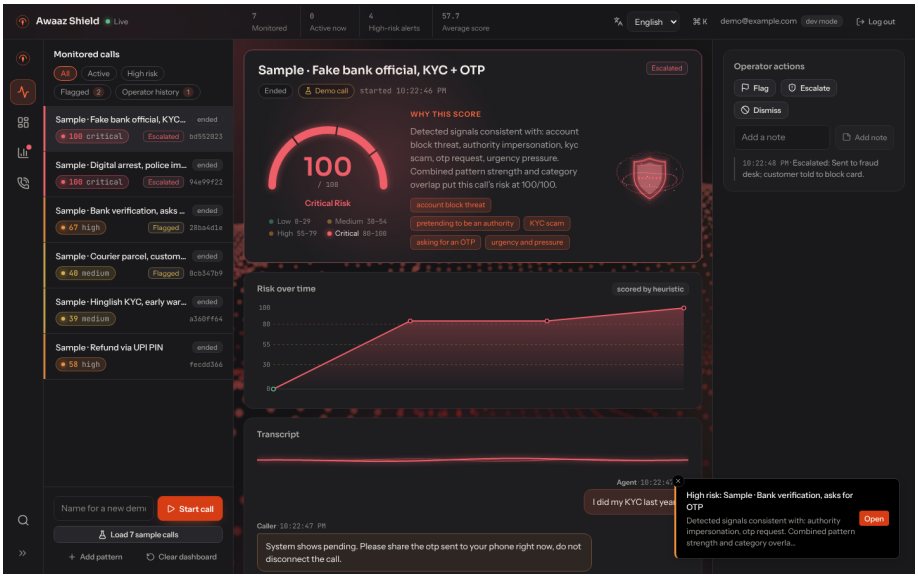


Figure 4. A critical call in the new design: the gauge, tactic chips, glowing 3D shield, risk chart and operator actions. The background has shifted to red.

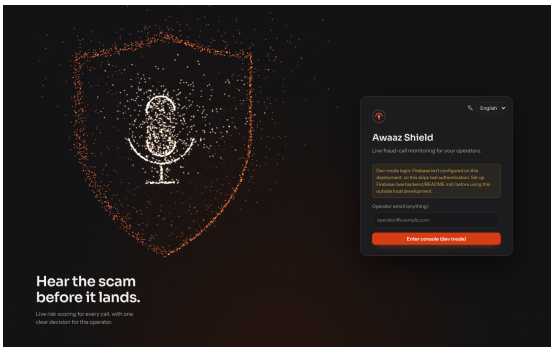


Figure 5. The login screen with the recoloured particle shield.

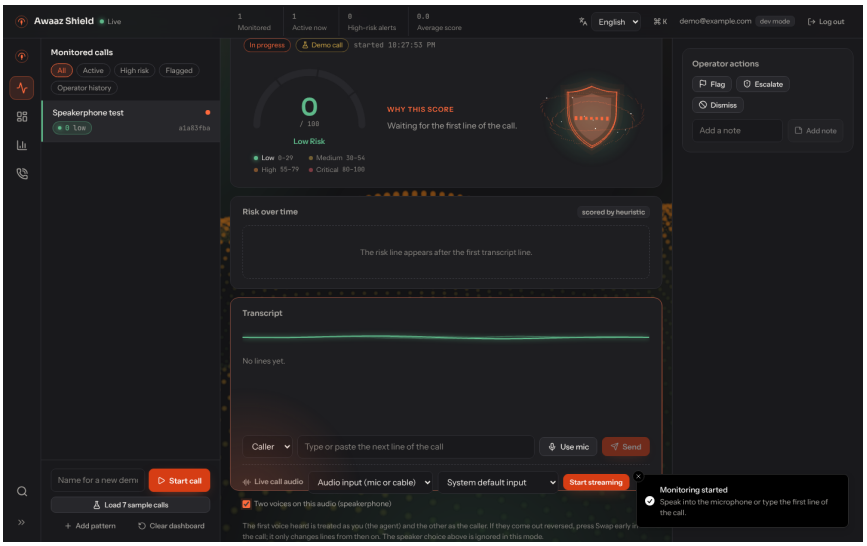


Figure 6. The Live call audio panel with the new two-voices option and hint (the language picker was added after this screenshot).

5. Risk score accuracy

The scorer was run on 55 labelled calls written in English and Hinglish (scams, normal bank calls, warnings, ambiguous calls), one line at a time, as the live app works.

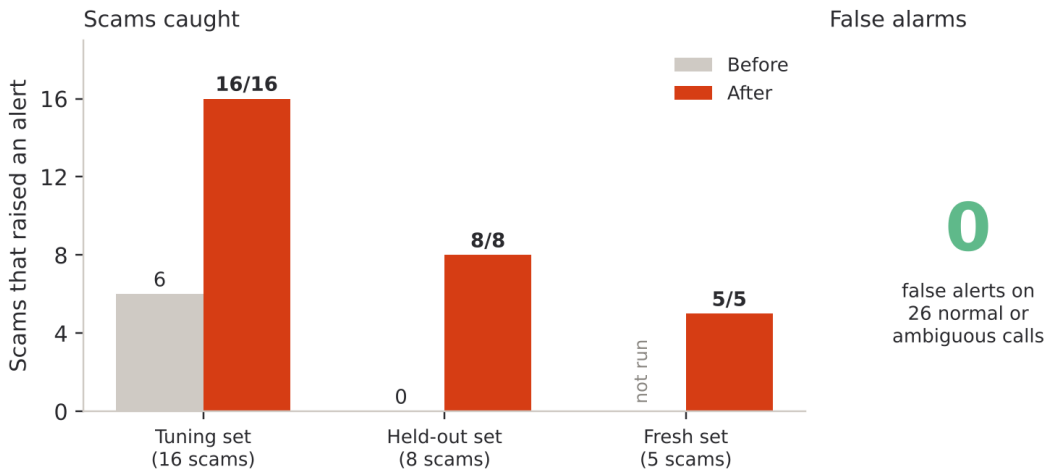


Figure 7. Scams that raised an alert before and after the scorer was rewritten, and false alerts on normal calls.

Measure	Before	After
Scams that raised an alert, tuning set (16)	6	16
Scams that raised an alert, held-out set (8)	0	8
Scams that raised an alert, fresh set (5)	not run	5
False alerts on normal or ambiguous calls (26)	0 of 21 measured	0

Measure	Before	After
Realistic speech-style set (re-run today)	-	10 of 10 scams alerted, 0 of 10 false alerts

Honest limits. The examples are hand-written, not recordings of real calls. Rules were adjusted after some failures, so the held-out and fresh sets are not fully clean. Expect misses on wording the scorer has not seen. Treat the score as help for a human operator, not a verdict.

6. Testing

Checked in this round

Check	Result
Backend smoke test (login, calls, scoring, alerts, actions, restore)	Passed
Persistence test (12 threads sharing one Firebase client)	Passed
Voice relay test (two-person in-app call, consent, audio forwarding)	Passed
Live audio relay test against a stand-in Deepgram, including new tests for speaker separation, swap, keyterms, language choice and phrase timing	Passed
Risk score evaluation (55 calls plus a realistic speech-style set)	Passed
Frontend unit tests	Passed (33 of 33)
Frontend production build	Passed
Headless-browser run against the real backend: login, consent, dashboard, sample calls, critical and medium calls, Insights, Command center, phone-width layout, live-audio panel	Passed, no console errors
Your own live call test (laptop microphone)	Call worked and risk was judged well; transcript text not very accurate

Not tested yet

Item	Why it matters
Transcript accuracy with the new settings (keyterms, English India, noise suppression, 0.5 s phrase timing)	They are the planned fix for your accuracy problem but need real audio to judge.
Speaker separation on a real speakerphone call	Separation on a laptop microphone will not be perfect, especially with noise or overlapping speech.
Two-person in-app call on two real phones	Needs a secure (https) address for phone microphones, for example a free tunnel such as cloudflared or ngrok.
Mac and Windows one-command start, audio cable setup, Chrome permission prompts	Tests ran in a Linux sandbox, not on your Mac. The new 3D and glow effects were checked in a software-rendered browser, not on a real GPU.
Real Firebase sign-in and the verification email	Needs a real attempt from you.
Claude scoring	Needs an Anthropic key, which was not provided.

7. How to run and configure

Run

- Open a terminal in the project folder and run **npm run dev**. Open the address it prints (normally http://localhost:5173). It starts the backend and frontend together.
- Run all backend and frontend checks with **npm test**.
- To test a live speakerphone call: start a call, pick the microphone, tick "Two voices on this audio", press Start streaming, and read a two-voice script aloud with the agent speaking first.

- To use two phones: put a free https tunnel in front of the frontend, add its address to the backend's allowed origins, then host and guest join with the call code.

Settings (backend/.env)

Setting	Purpose	Default
DEEPGRAM_API_KEY	Turns on live speech to text. Stays on the server.	Set by you
DEEPGRAM_MODEL	Speech model.	nova-3
DEEPGRAM_LANGUAGE	Server-wide recognition language: multi (English plus Hindi), en-IN, en or hi.	multi
DEEPGRAM_KEYTERMS	Comma-separated words to listen for harder. Empty turns it off.	Built-in scam-call list
DEEPGRAM_ENDPOINTING_MS	Silence that closes a phrase. Higher keeps sentences whole but adds delay.	500
ANTHROPIC_API_KEY	Optional: Claude scoring in place of the rules engine.	Empty

8. Known limits and risks

Item	Detail
No carrier integration	Only typed lines, the microphone, a virtual cable, a shared tab and the in-app call feed the app. There is no phone-network integration.
Scorer is rules-based	Its accuracy figures come from examples the same author wrote and tuned against. Real call transcripts are needed to measure it properly.
Firebase key shared in chat	The service-account key was shared in a chat. Rotate it.
Consent	Get consent before recording or transcribing anyone's call. The app asks both people in an in-app call to accept a notice first.
Speaker labels	In speakerphone mode, labels can come out reversed. Swap only changes lines from then on, and earlier lines and their scores are not recalculated.
Visual effects on weak devices	The shield, wave and glow effects are heavier than a plain design. They pause when hidden and switch off with the reduced-motion setting, but were not tested on slow hardware.
Language	Hindi in Devanagari script is only partly covered by the scorer, so mixed Hindi and English should use the automatic language setting. The new panel text is shown in English in the other languages.

9. Next steps

1. **Test the speakerphone call with real audio.** Run the English and Hinglish scripts. Send me a few lines the transcript got wrong, and say whether it was English, Hindi or Hinglish.
2. **Test the in-app call on two phones.** Use the free https tunnel. This is the route that gives clean caller and agent separation.
3. **Rotate the Firebase service-account key.** It was shared in a chat.
4. **Collect real or realistic transcripts.** Add them to backend/eval/risk_cases.py. This is the best way to keep improving the score.
5. **Tune vocabulary and language from what you find.** Edit DEEPGRAM_KEYTERMS or the language choice based on the errors.
6. **Turn it into an app.** The project already has an install manifest and service worker, so adding it to a phone's home screen is the first step before any native app.